ARM

Dr. KBR

→ ARM was developed at Acorn computer limited of Cambridge England between 1983 and 1985.

→ RISC concepts were Introduced in 1980 at Stanford and Berkeley

→ Acorn started in 1983 and by 1985 design of first commercial RISC machine called Acorn RISC Machine (ARM), later it is called "Advanced RISC Machine".

→ ARM cortex core
- Cortex-A: Application processor core
- Cortex-R: Real-time applications core
- Cortex-M: Microcontroller cores for a wide range of applications.

NOTE: ARM7 implements both ARM and Thumb instruction sets. Cortex-M3 supports only the Thumb-2 Instruction set

Thumb-2: Thumb-2 is an enhancement to the 16-bit Thumb instruction set. It adds 32-bit Instructions that can be freely intermixed with 16-bit instructions in a program.

* STM32F407VGT6: ST Microelectronics
- Manufacturer: ST Microelectronics
- product category: ARM Microcontrollers
- Series: STM32F407VG
- Core: ARM cortex M4
- program Memory size: 1MB
- Data Bus width: 32 bit
- ADC Resolution: 12 bit
- Maximum clock frequency: 168 MHz
- Number of I/Os: 82 I/o
- Data RAM Size: 192 KB

Data RAM Type: SRAM

Interface type: CAN, I2C, SDIO, I2S/SPI, UART/USART, USB.

* STM32F4 ⟶ Discovery board

* STM32 CubeMx ⟶ is a graphical tool that allows a very easy configuration of STM32 microcontrollers and Microprocessors and generation of corresponding initialization "C" code for the Arm cortex-M core.

* STM32 ⟶ is a family of 32-bit microcontroller by ST Microelectronics. The STM32 chips are grouped into related series that are based around the same 32-bit ARM processor core.
Ex: cortex-M0, cortex-M0+, cortex-M3, cortex-M4, cortex-M7, cortex-M33.

* STM32 is much powerful than AVR. Industrial PLCs and other complex devices need the extra processing power and larger memory capacity.

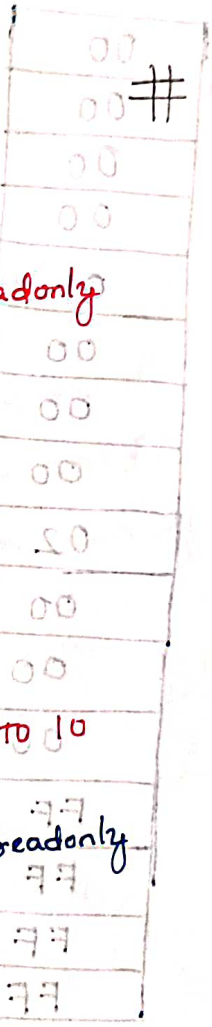
* The STM32F4 family incorporates high speed Embedded memories and an extensive range of Enhanced I/os and peripherals connected to two APB busses, three AHB busses and 32-bit multi-AHB bus matrix.

* Sample program:

space → space → press underscore twice

space export __main → must be lowercase letters

```
area prog3_2, code, readonly

__main
        mov R0, #0x23;
        MOV R1, #0x22;
        ADD R3, R0, R1;
here    b here
        end
```



* Sample program:

```
        export __main
        area prog1_2, code, readonly

__main  mov r0, #12
        add r0, r0, #4
here    b here
        end
```

* Sample program:

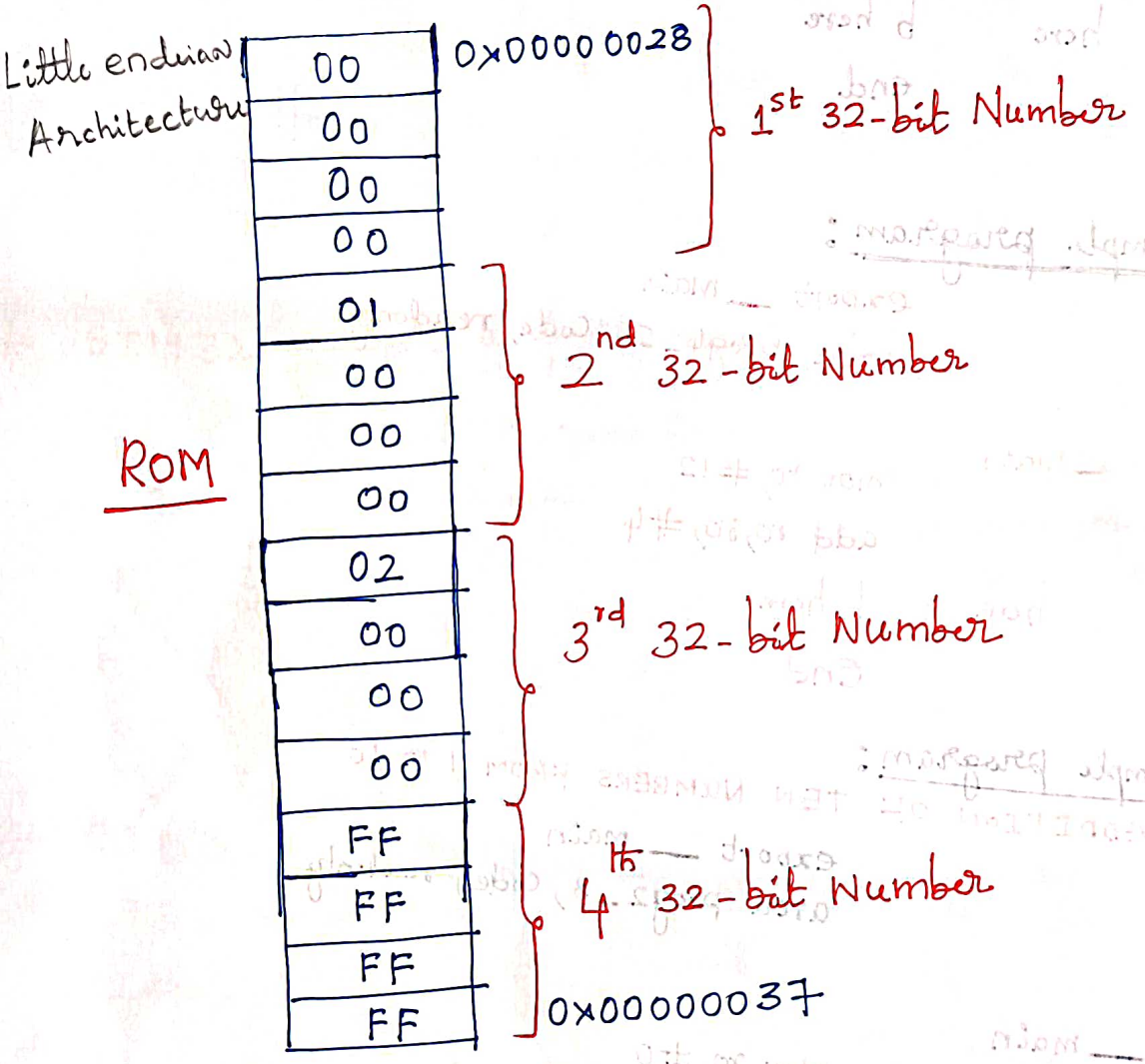
```
; ADDITION OF TEN NUMBERS FROM 1 TO 10
        export __main
        area prog2_3, code, readonly

__main  mov r0, #0
        mov r1, #1
loop1   add r0, r1
        add r1, #1
        cmp r1, #0x0b
        bne loop1
here    b here
        end
```

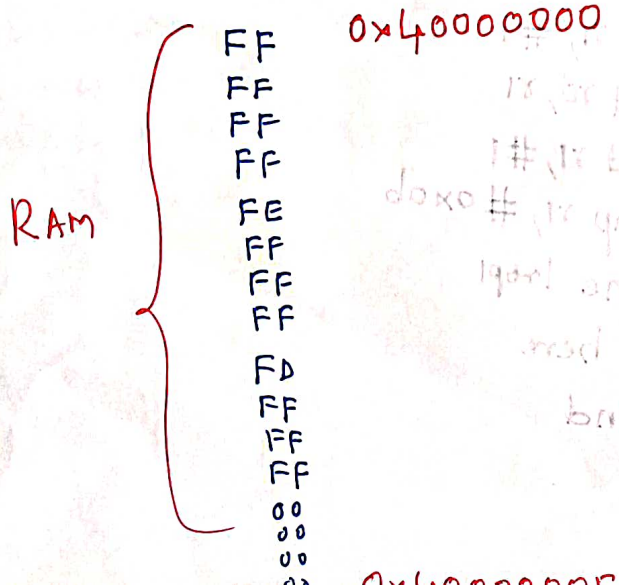
* Steps to be followed for program Execution:

1. Go to projects ⟶ new project
2. Select Device for Target ⟶ STM32F407VGT6

* How data 32-bit numbers are stored in ROM Memory

Data: 0, 1, 2, FFFFFF





```
        export __main
        area prog1, code
__main
        mov r0, #4
        ldr r1, =0x20000000
        ldr r2, =0x20000022
loop1   ldrdb r3, [r1], #1
        ; mvn r3, r3
        strb r3, [r2], #1
        subs r0, r0, #1
        cmp r0, #0
        bne loop1
here    b here
        ; area x, data, readonly
        ; dcb 0x0, 0x1, 0x2, 0xff
        ; area y, data, readwrite
        ;
        end
```

* Data Transfer Instructions Of ARM

Dr. KBR

↳ Normal Mov Instruction:

```
Mov R0, R1 ; copying the content of R1 into R0.
```

↳ Immediate data Mov Instruction

```
Mov R0, #0x25 ; Moving data 25h into R0 Register
```

NOTE: only 8-bits data can be moved Immediately.

↳ Normal Mov Instruction which affect the flag

```
Movs R0, R1 ; while moving the data which updates
              the status flag.
```

Example: If R1=0, then after Movs R0, R1 ; R0=1 and sets ZF=1 (Zero flag)

If R1 = is negative value, then after moving the data, sets negative flag, N=1.

↳ Conditional Mov Instruction: It performs move based on previous Instruction status.

```
Movcc R0, R1 ; It moves the data only if carry flag is zero
```

- cc — carry clear, C=0
- cs — Carry set, C=1
- VS → overflow set, V=1
- VC → overflow clear, V=0
- EQ → Equal, Z=1
- NE → Not equal, Z=0

```
Movcss R0, R1 ; It performs move based on previous instruction
                status and also it will affect the flag after
                execution
```

↳ It moves the data only if carry flag is set and after move operation, it updates the status flag.

↳ MOV NOT Instruction: It performs move with NOT operation

MVN R0, R1

↳ with MVN Instruction we can use condition as well as status

Implementation

MVNVSS R0, R1

* Write assembly program to transfer block of 4 word (16 bytes) of data from one block of memory to another block of memory

```
        export __main
        area prog1, code

__main  mov r0, #4
        ldr r1, =0x20000000      ; ADR R1, x ;
        ldr r2, =0x20000022
; ldr R1, =x
  ldr r2, =y

loop1   ldrb r3, [r1], #1        ; R3 = ~R3
        ; mvn r3, r3
        strb r3, [r2], #1
ldr r3, [r1], #4;
str r3, [r1], #4 ;
Subs r0, r0, #1
        Subs r0, r0, #1          ; Modify status flag after the operation.
        cmp r0, #0
        bne loop1

here    b here

        ; area x, data, readonly
        ; dcb 0x0, 0x1, 0x2, 0xff
Define constant byte (DCB)
Define constant word (DCW)
Define constant double word
        ; area y, data, readwrite
        end
```

```
x dcd 0, 1, 2, 0xffffffff
        area data1, data, readwrite
y dcd 0, 0, 0, 0
        end
```

```
export __main
area prog1, code, readonly

__main
        ldr r1, =x
        ldr r2, =y
        ldr r3, [r1]
        str r3, [r2]
here    b here

        ; area x, data, readonly
        ; dcd 0x0ffffffe
        ; area y, data, readwrite
        ; dcd 0x00000000

        area data1, data, readonly
x dcd 0xffffccdd
        area data2, data, readwrite
y dcd 0
        end
```

```
export __main
area prog1, code, readonly

__main
        ldr r1, =0x20000000
        ldr r2, =0x20000020
        ; ldr r3, [r1]
        ; ldrh r3, [r1]
        ldrb r3, [r1]
        mov r4, r3
        ; str r3, [r2]
        ; strh r3, [r2]   ; loads a half-word
        strb r3, [r2]
here    b here

        end
```

* Simple Data transfer operation

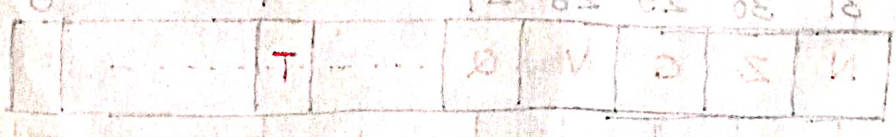
```
; Simple Data transfer operation
    export __main
    area prog2, code

__main
    mov r1, #0x25 ;
    mov r2, r1
    mov r3, r1
    ldr r4, =0x20000000
    Str r1, [r4]
here  b here
    end
```

```
; Simple Data transfer operation
    export __main
    area prog3, code

__main
    mov r0, #1 ;      r0 = 0x00000001
    mov r1, #0x21 ;   r0 = 0x00000021
    mov r2, #21 ;     r2 = 0x00000015
    mov r3, r1 ;      r3 = 0x00000021
    MVN r0, #0xff ;   r0 = 0xffffff00
    MVN r0, r3 ;      r0 = ~r3 = 0xffffffde
    Ldr r0, [r1] ; Load r0 with the contents
    Str r0, [r1] ; of the memory address
                 ; specified in r1.
here  b here
    end
```

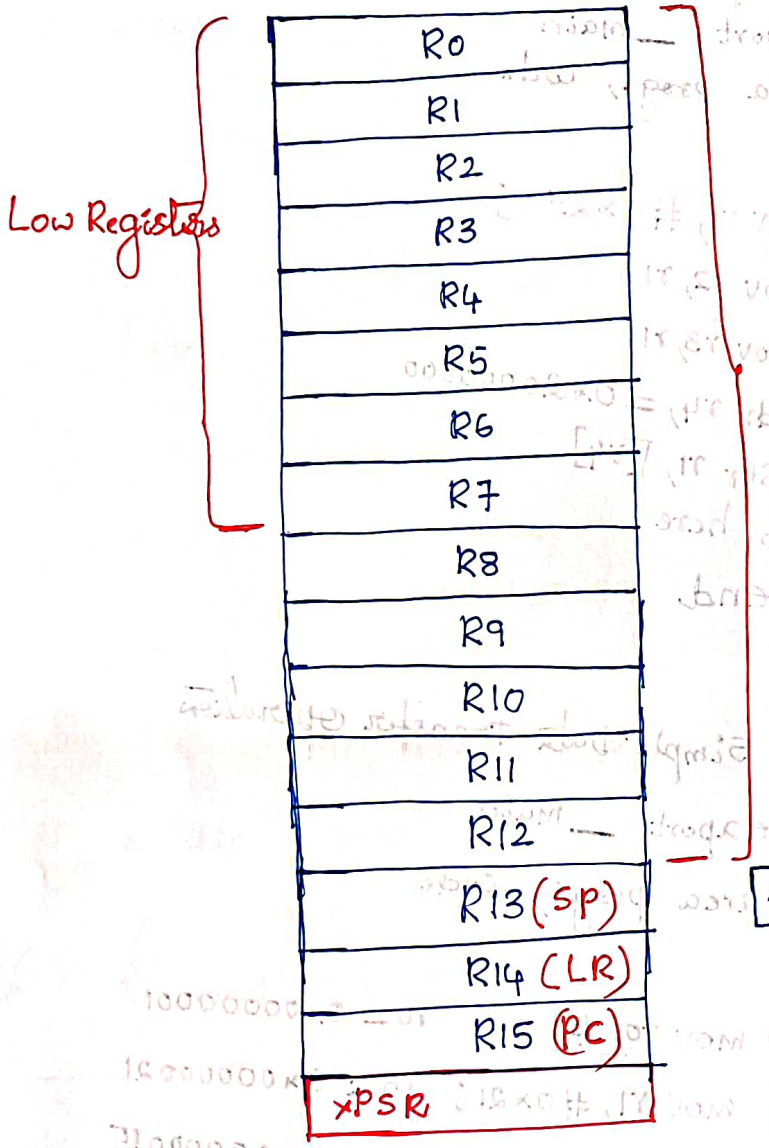
16|21
1-5

$(21)_{10} = 15H$

0010 0001
1101 1110
d e



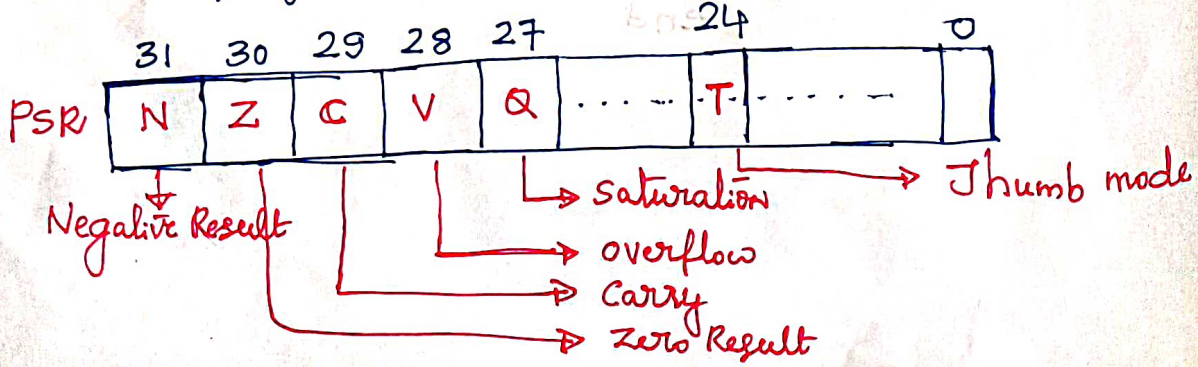
* Registers for ARM cortex M4 (M series): 'programmers model'



* SP → Stack pointer
* R14 (LR) → Link Register
* PC (R15) → program counter
* PSR → program status Register

* Thee status Registers:

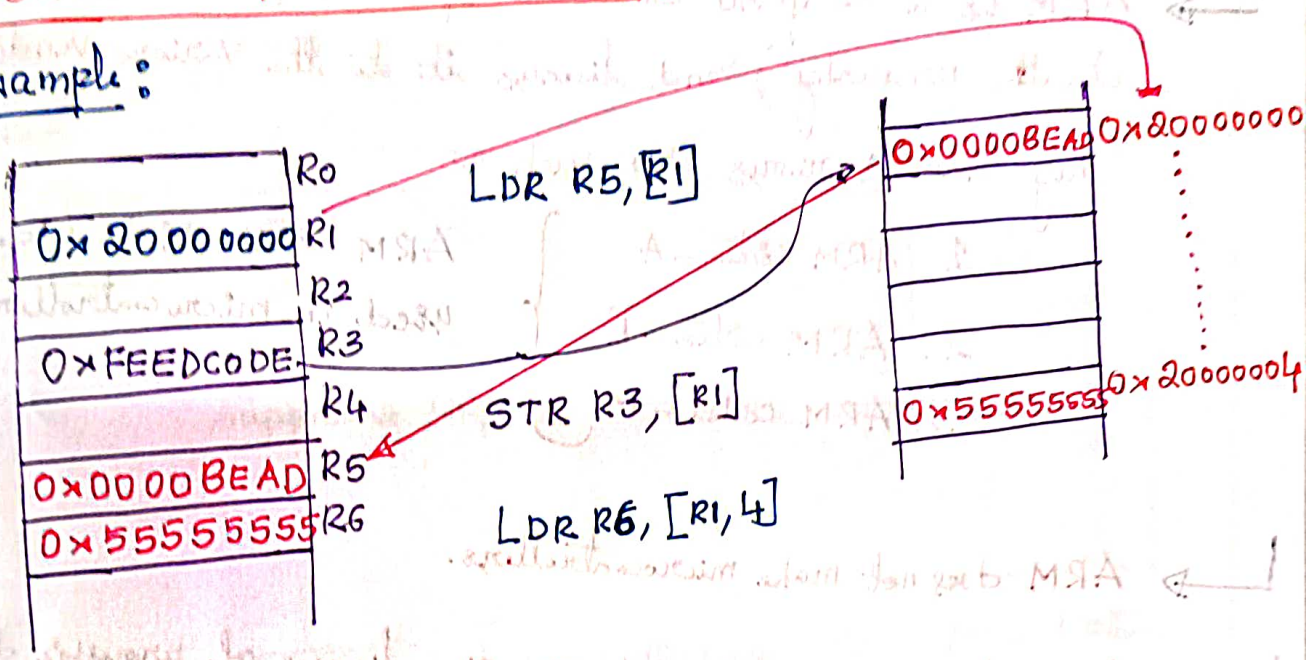
1. Application program status Register (APSR)
2. Interrupt program status Register (IPSR)
3. Execution program status Register (EPSR)



NOTE: These Registers APSR/IPSR/EPSR can be accessed all individually or as a PSR (together)

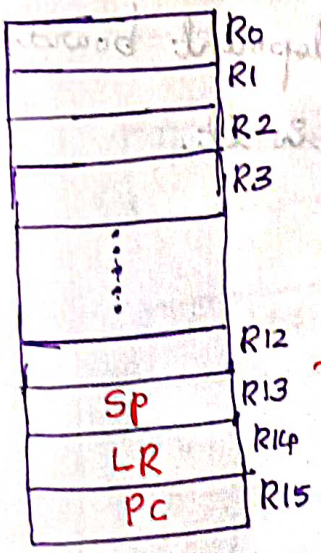
* Basic Load/Store Instructions:

Example:



* Where the processor/microcontroller stores or obtains Information

- Registers
- Memory
- Input/output Devices



* Sixteen generic 32-bit Registers
  - Thirteen are for general purposes
    - can hold data or address
    - Data may be byte, halfword, or word

Ex:
```
ldr r3, [r1]  ; 32 bit
ldrh r3, [r1] ; halfword/16-bit
ldrb r3, [r1] ; 8 bits/byte

str r3, [r1]
strh r3, [r2]
strb r3, [r2]
```

* NOTE :

↳ ARM is a microprocessor and it is used inside ARM-based microcontroller.

↳ ARM is a company which manufactures processor IP ("Intellectually property") and licenses it to the various vendors. They have various cores such as

1. ARM cortex-A
2. ARM cortex-R
3. ARM cortex-M

} ARM cortex-M is widely used in microcontrollers.

↳ ARM does not make microcontrollers.

↳ ARM designs processors and give licenses of processor designs to various microcontroller vendors. Typically we call these designs "Intellectually property" (IP) and business model is called IP licensing.

↳ * Is Arduino a microprocessor or a microcontroller ?

↳ Arduino is not a microcontroller nor a microprocessor. It's a simple and easy-to-use development board that is relying on a microcontroller in it.

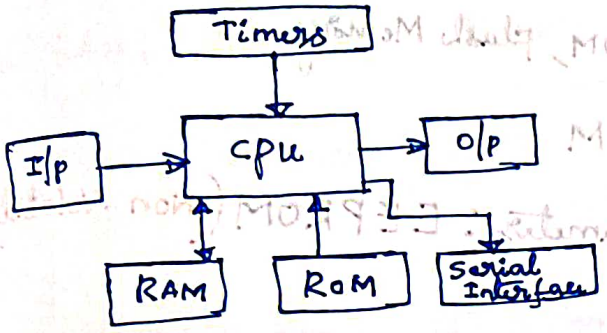
↳ Simple microcontroller Board

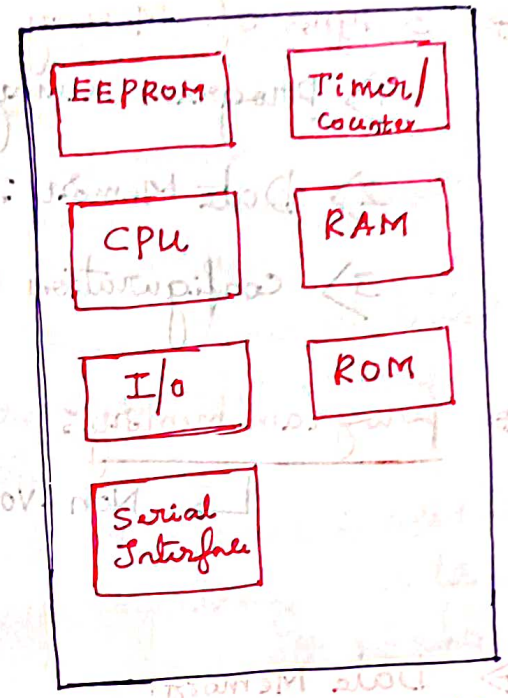
* ARM is a company who license cpu cores.

* ARM is core for both microprocessors and micro-controller

Microcontrollers and programming

Microprocessor Vs Microcontroller



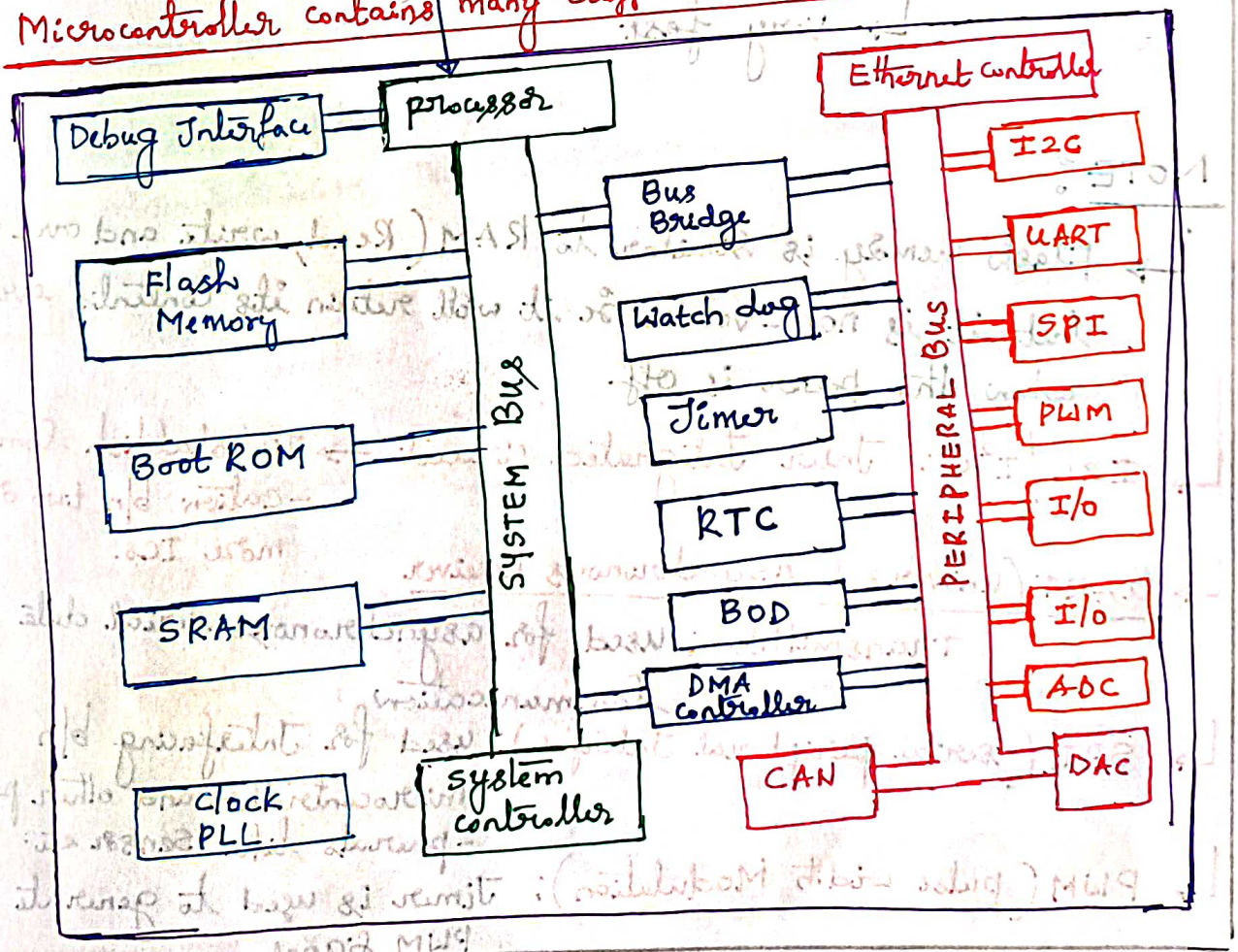


Microprocessor: CPU and several supporting chips

Microcontroller: CPU and other peripherals on a single chip.

ARM cortex – M processor

* **Microcontroller contains many different Blocks:**



Dr. KBR

* Microcontroller Memories:

↳ 3 types of Memory

1) program memory: ROM, Flash Memory
2) Data Memory: RAM
3) configuration parameters: EEPROM (Non-Volatile)

→ **program memory:**

↳ Non-Volatile: Don't lost data when shut down power

→ **Data Memory:**

↳ Volatile: Data is lost when shut down power

↳ Store variables during program execution

↳ very fast

NOTE:

↳ Flash memory is similar to RAM (Read, write and over write) but it is non-volatile so it will retain its contents even when the power is off

↳ I2C / I^2C: Inter Integrated circuit → TO establish communi-cation b/n two or more ICs.

↳ UART (universal Asynchronous Receiver transmitter): used for asynchronous serial data communication

↳ SPI (serial peripheral Interface): used for Interfacing b/n microcontroller and other peri-pherals like Sensor etc.

↳ PWM (pulse width Modulation): Timer is used to generate PWM signal

Dr. KBR

↳ RTC (Real time clock): Keep track of accurate time. Consists of oscillator circuit and Quartz crystal

↳ CAN (Controller Area network): It's a protocol for communication purposes

↳ BOD (Brown-out detection): BOD circuit monitors the supply voltage which compares supply voltage to fixed trigger level

↳ Watchdog timer: It's a simple count down timer which is used to reset a microprocessor after a specific interval of time.

↳ PLL clock (phase-locked loop) clock: This circuit is used to synchronize oscillator signal with reference signal

↳ Timer: is a circuit used to measure time Intervals.

* Cortex-M3 | M4 microcontroller Vendors:

↳ Analog devices, Atmel, cypress, Infineon, Samsung, Silicon labs, ST Microelectronics, Texas Instruments, Toshiba etc.

NOTE:

↳ After a company licenses the cortex-M processor design, ARM provides the design source code of the processor in a language called Verilog-HDL (Hardware description Language).

↳ ARM cortex → is a group of 32-bit RISC ARM processor cores, licensed by ARM limited
→ series of cores

Dr. KBR

* Advantages Of the cortex® - M processors:

1. **Low power:** The cortex - M processor designs are optimized in a such a way that designs are relatively small compared to other 32-bit designs. Many cortex - M microcontrollers have power consumption of less than 200 µA/MHz and some microcontrollers having < 100 µA/MHz.
2. **performance:** Handles or executes more complex and demanding applications
3. **Energy efficiency:** Combining low power and high-performance characteristics, having excellent Energy -efficiency.
4. **Code density:** The Thumb ISA (Instruction set Architecture) provides excellent code density.
5. **Interrupts:** It supports up to 240 Vectored Interrupts and multiple levels of Interrupt priorities (from 8 to 256 levels). It supports nesting of Interrupts. This feature makes the cortex - M processors suitable for real-time control applications
6. **Easy programming using 'C' Language:** Having simple linear memory mapping no architectural restrics.
7. **Easy scalability:** Easy scaling of designs from low cost simple designs to high cost multi processor designs.
8. **Debug features:** allows the user to analyze design problems easily. It provides halt, single step, trace to capture program flow, data changes, profiling Information and so on.
9. **OS support:** supports OS applications, cwrently there are over 30 embedded OSs are available for cortex - M processors.
10. **Versatile System features:** Bit addressable memory range (Bit band feature) and MPU (Memory protection unit)

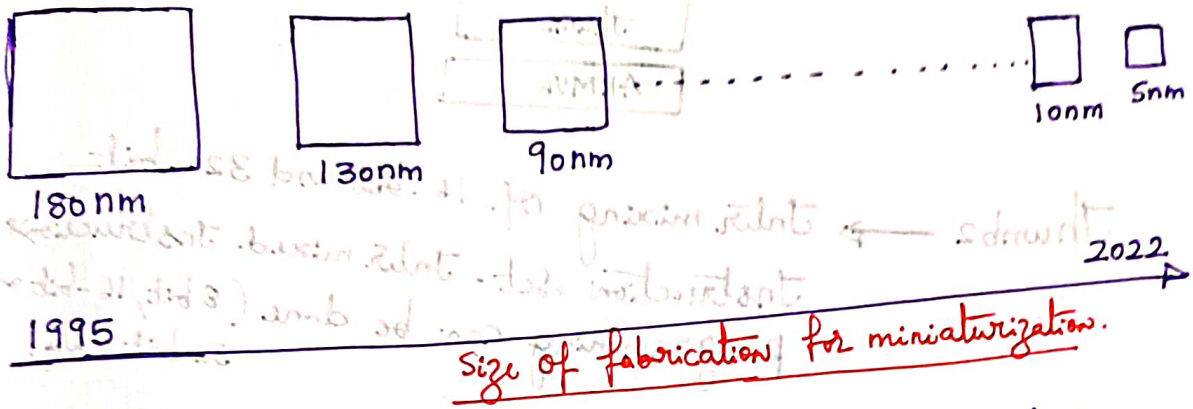
Dr. KBR

11. **Software portability and Reusability:** CMSIS (cortex microcontroller Software Interface standard) provides standard header files and APIs. This allows better software reusability and also porting applications code easier.

12. **Choices (Tools, devices, OS, etc):**

Dr. KBR

* Development History of ARM processor / ARM processor Evolution



↳ The first ARM processor (ARM1) was developed at Acorn computers limited by a team led by Sophie wilson and Steve furber.

↳ Joint venture between Acorn computers, apple and VLSI Technology. Acorn provided 12 employes, VLSI provided tools and Apple provided $3 million Investment

↳ ARM processor uses RISC Architecture

↳ ARM company doesnot make different applications.

↳ ARM company makes cpu cores

↳ ARM company provides cpu cores to different companies

↳ Different Vendors uses ARM cpu core and develops different applications according to their needs.

1) ARM7TDMI → Thumb, ARMv4 → 16 bit Instruction set, 32 bit Instruction set

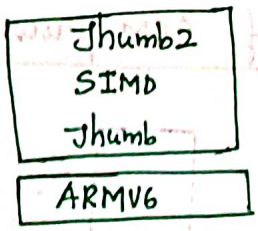
Instruction set (TWO types of Instruction sets)

1) ARM state Instruction set: used for higher category of applications

2) Thumb state Instruction set: used for lower category of applications

2) ARM9 → Added some more Instructions set features like execution of Java Instructions

3) ARM11 →



Thumb2 → Inter mixing of 16-bit and 32-bit Instruction set. Inter mixed Instructions programming can be done (8 bit, 16-bit and 32 bit data)

4) ARM cortex family

ARM cortex →
- A: Designed for high performance open application platforms.
- R: Designed for high end Real-time embedded systems
- M: Microcontroller based embedded systems